

Avoid template bloat for `safe_ranges` in combination with ‘`subrange-y`’ view adaptors.

Document #: P1739R3
Date: 2019-11-24
Project: Programming Language C++
Library Working Group
Reply-to: Hannes Hauswedell
<<h2 AT fsfe.org>>

Contents

- [1 Revision History](#)
 - [1.1 R3](#)
 - [1.2 R2](#)
 - [1.3 R1](#)
 - [1.4 R0](#)
- [2 Introduction](#)
- [3 Motivation and Scope](#)
- [4 Impact on the Standard](#)
- [5 Design Decisions](#)
 - [5.1 Possible objections to this proposal](#)
 - [5.2 Stronger proposals](#)
- [6 Technical Specifications](#)
 - [6.1 Changes to `views::take` \(§24.7.6.4\):](#)
 - [6.2 Changes to `views::drop` \(§24.7.8.3\):](#)
 - [6.3 Changes to `views::counted` \(§24.7.12\):](#)
 - [6.4 Changes to `iota\_view` \(§24.6.3\):](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Revision History

§ 1.1 R3

1. R2 of this paper was accepted (again) by LEWG in Belfast, but subsumed by P1664 which was also discussed in Belfast and accepted by both LEWG and LWG. P1664 was due to be voted on as Motion 14 but withdrawn before the poll.
2. The concern was that views providing “reconstructible” interfaces (e.g. construction from iterator-sentinel-pair) may still provide deviating semantics so there should be an opt-in or opt-out mechanism from the concepts.
3. Due to the very late state of the C++20 standardisation, we decided to uphold P1739 with the bare minimum of necessary changes that would otherwise be breaking. These include hard-coding the behaviour for the currently affected standard library types.
4. Any generalisation to user-defined types or the definition of concepts (P1664) is postponed to after C++20. Using an opt-in trait in the future means that none such changes would be breaking.
5. The wording changes for `empty_view` (introduction of NOP-constructor) were removed, because `empty_view` is treated separately for now. The wording changes by P1664 for `iota_view` were imported except the changes which are already adopted via P1870 (`safe_range` / `forwarding_range`).
6. The current paper is very close to the original intent and wording of P1739.

Only the wording changes have been updated, not the body of the paper.

§ 1.2 R2

1. Change the name of the paper to reflect more accurately that no type erasure happens (in contrast to the earliest versions of this paper).
2. Updated standard references to the CD; track naming changes.
3. Provide html, because markdown isn’t rendered automatically.

§ 1.3 R1

1. In the changes for `views::take` and `views::drop`, also consider constructors of the input type that take iterator and sentinel as arguments (instead of just a constructor that takes a subrange over the two). This enables us to act on such types that don’t provide the range-constructor (e.g. unclear for `basic_string_view`, in general single-argument constructors are unpopular...). If both constructors are available we prefer the (iterator, sentinel)-constructor to avoid unnecessarily instantiating the subrange-template.
2. We have added such an (iterator, sentinel)-constructor to `ranges::empty view` so that it becomes one of the types that benefit from the changes to `views::take` and `views::drop`.

These changes were approved by LEWG as part of the discussion of R0 in Cologne.

§ 1.4 R0

First revision.

§ 2 Introduction

The current draft standard introduces many range adaptor objects [24.7]. Upon invocation, most of these return a type that is specified together with the adaptor, e.g. `views::take` returns a specialization of `ranges::take_view`. Chaining multiple adaptors thus leads to an increasingly nested type. This proposal suggests avoiding nested return types when the input is a view that already represents a “subrange” and it suffices to “change the bounds”.

§ 3 Motivation and Scope

Given the following example:

```
vector vec{1, 2, 3, 4, 5};
span s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
          | views::drop(1) | views::take(10);
```

The type of `v` will be something similar to

```
ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int,
dynamic_extent> > > > >
```

Although in fact it could just be `span<int, dynamic_extent>`, i.e. the same as the original type of `s`. This is also true for other input types that are “subrange-y” and it would feel natural to preserve the original type if only the size of the “subrange” is changed, i.e. `s | views::drop(1) | views::take(3)` should be equivalent to `s.subspan(1, 3)` if `s` is a `span`.

We propose that `views::drop` and `views::take` return a range of the same type as the input type (and not return a nested type) if the input is a *forwarding-range* that can be constructed from its own (adjusted) iterator and sentinel (or a `ranges::subrange` over the iterator-sentinel-pair). This includes:

- a `span` of dynamic extent (proposed by P1394)
- a `basic_string_view` (proposed by P1391)
- a `ranges::subrange` that models `ranges::RandomAccessRange` and `ranges::SizedRange`
- a `ranges::empty_view` (proposed in this paper)

We also propose that the `views::counted` range factory return a span of dynamic extent (instead of `ranges::subrange`) if the iterator passed to it models `ContiguousIterator`.

The proposed changes will strongly reduce the amount of template instantiation in situations where `views::drop` and `views::take` are applied repeatedly. They increase the integration of the view machinery from `ranges` with the views created in the standard independently (`span` and `basic_string_view`). And they improve the teachability and learnability of views in general, because some of the simplest view operations now return simpler types and behave as the already documented member functions.

§ 4 Impact on the Standard

This proposal includes changes to the current draft standard. All proposed changes are library changes and none of the proposed changes affect the published international standard.

The proposal targets C++20, because applying these changes afterwards would be breaking.

The currently proposed wording changes depend on the following proposals:

- [P1391](#) Range constructor for `std::string_view`
- [P1394](#) Range constructor for `std::span`

Although the wording could be adapted to handle the mentioned types specifically – if the respective papers are not accepted (in time).

[P1664](#) defines concepts that this proposal could use to specify its behaviour more elegantly. It generalises the notion of `reconstructible-ranges` and makes more ranges “reconstructible” that can then benefit from the optimisations/fixes described here.

§ 5 Design Decisions

§ 5.1 Possible objections to this proposal

Following this proposal means that it will not hold any longer that “*the expression `views::take(E, F)` is expression-equivalent to `take_view{E, F}`”*, i.e. the adaptor has behaviour that is distinct from the associated view class template. This may be a source of confusion, however by design of the current view machinery users are expected to interact with “the view” almost exclusively through the adaptor/factory object. And this is not the first proposal to define such specialised behaviour of an adaptor depending on its input: [P1252](#) (accepted) modified `views::reverse` to “undo” the previous reversal when passed a type that already is a specialisation of `ranges::reverse_view` (instead of “applying” it a second time). The added flexibility of the adaptor approach should be seen as a feature.

While specialisations of e.g. `ranges::take_view` provide a `base()` member function that allows accessing the underlying range, this is not true for `span` or the other return types that we are proposing. However, since the underlying range is of the same type as the returned range, we see little value in re-transforming the range to its original type. It should also be noted that this “feature” (a `base()` member function) is provided only by some of the views in the draft standard and is not a part of the general view design. It cannot be relied on in generic contexts and combinations with other views and no parts of the standard currently make use of it.

§ 5.2 Stronger proposals

This proposal originally included also changing `views::all` to type-erase `basic_string` constants to `basic_string_view`; all forwarding, contiguous ranges to `span`; and all forwarding, sized, random-access ranges to `ranges::subrange`. `views::all` is the “entry-point” for all non-views in a series of view operations and normally wraps non-view-ranges in `ranges::ref_view`. Changing `views::all` in this manner would have a type-erasing effect, e.g. `s | views::drop(1) | views::take(3)` would return a `span`, independent of whether `s` is a `span`, a `vector` or an `array`.

This form of type erasure would preserve all range concepts currently defined. It is, however, possible that future more refined concepts would no longer be modelled by an input type erased in the above fashion. For this reason Casey Carter argued against changing `views::all` and it was dropped from this proposal.

[The current proposal does not suffer from this uncertainty, because we are preserving the exact input type and there is no erasure.]

§ 6 Technical Specifications

§ 6.1 Changes to `views::take` (§24.7.6.4):

```
The name views::take denotes a range adaptor object.
- For some subexpressions E and F, the expression views::take(E, F) is
  expression-equivalent to take_view{E, F}.
+Let E and F be expressions, and let T be remove_cvref_t<decltype((E))>.
+The expression views::take(E, F) is expression-equivalent to:
+  - T{} if T is a specialization of ranges::empty_view
    ([range.empty.view]).
+  - Otherwise, T{ranges::begin(E), ranges::begin(E) + min<range_
    difference_t<decltype((E))>>(ranges::size(E), F)}
+    if T is a specialization of one of the following class templates and
    except that E is only evaluated once:
+      1. span ([views.span]) of dynamic extent;
+      2. basic_string_view ([string.view]);
+      3. ranges::iota_view ([range.iota.view]) if T also models
        ranges::sized_range;
+      4. ranges::subrange ([range.subrange]) if T also models
        ranges::random_access_range and ranges::sized_range.
+  - Otherwise, ranges::take_view{E, F}.
```

§ 6.2 Changes to `views::drop` (§24.7.8.3):

The name `views::drop` denotes a range adaptor object.

- For some subexpressions `E` and `F`, the expression `views::drop(E, F)` is expression-equivalent to `drop_view{E, F}`.

+ Let `E` and `F` be expressions, and let `T` be `remove_cvref_t<decltype((E))>`.

+ The expression `views::drop(E, F)` is expression-equivalent to:

- + - `T{}` if `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`).
- + - Otherwise, `T{ranges::begin(E) + min<range_difference_t<decltype((E))>>(ranges::size(E), F), ranges::end(E)}` if `T` is a specialization of one of the following class templates and except that `E` is only evaluated once:
 - + 1. `span` (`[views.span]`) of dynamic extent;
 - + 2. `basic_string_view` (`[string.view]`);
 - + 3. `ranges::iota_view` (`[range.iota.view]`) if `T` also models `ranges::sized_range`;
 - + 4. `ranges::subrange` (`[range.subrange]`) if `T` also models `ranges::random_access_range` and `ranges::sized_range`.
- + - Otherwise, `ranges::drop_view{E, F}`.

§ 6.3 Changes to `views::counted` (§24.7.12):

The name `views::counted` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, and let `T` be `decay_t<decltype((E))>`. Then the expression `views::counted(E, F)` is expression-equivalent to:

- + - If `T` models `input_or_output_iterator` and `decltype((F))` models `convertible_to<iter_difference_t<T>>`,
- + - `span{to_address(E), static_cast<iter_difference_t<T>>(F)}` if `T` models `contiguous_iterator`.
- - `subrange{E, E + static_cast<iter_difference_t<T>>(F)}` if `T` models `random_access_iterator`.
- + - Otherwise, `subrange{E, E + static_cast<iter_difference_t<T>>(F)}` if `T` models `random_access_iterator`, except that `E` is only evaluated once .
- - Otherwise, `subrange{counted_iterator{E, F}, default_sentinel}`.
- - Otherwise, `views::counted(E, F)` is ill-formed.

§ 6.4 Changes to `iota_view` (§24.6.3):

The following changes make `ranges::iota_view` constructible from its iterator-sentinel-pair.

In §24.6.3.2:

```
namespace std::ranges {
    template<class I>
        concept decrementable =          // exposition only
            see below;
    template<class I>
        concept advanceable =            // exposition only
            see below;
```

```

template<weakly_incrementable W, semiregular Bound =
unreachable_sentinel_t>
    requires weakly-equality-comparable-with<W, Bound>
class iota_view : public view_interface<iota_view<W, Bound>> {
private:
    // [range.iota.iterator], class iota_view::iterator
    struct iterator;           // exposition only
    // [range.iota.sentinel], class iota_view::sentinel
    struct sentinel;          // exposition only
    W value_ = W();           // exposition only
    Bound bound_ = Bound();    // exposition only
public:
    iota_view() = default;
    constexpr explicit iota_view(W value);
    constexpr iota_view(type_identity_t<W> value,
                        type_identity_t<Bound> bound);

+   constexpr iota_view(iterator first, sentinel last) : iota_view(*first,
+   last.bound_) {}

    constexpr iterator begin() const;
    constexpr auto end() const;
    constexpr iterator end() const requires same_as<W, Bound>;

    constexpr auto size() const requires see below;
};

template<class W, class Bound>
    requires (!is-integer-like<W> || !is-integer-like<Bound> ||
              (is-signed-integer-like<W> == is-signed-integer-like<Bound>))
    iota_view(W, Bound) -> iota_view<W, Bound>;
}

```

§ 7 Acknowledgements

Thanks to Casey Carter, Eric Niebler, Barry Revzin and Corentin Jabot for feedback on the proposal. Many thanks to JeanHeyd Meneide for alerting me to and involving me in P1664 which would solve some of the things described here more elegantly (and more generically).

§ 8 References

- [P1252](#) Input range adaptors
- [P1035](#) Ranges Design Cleanup
- [P1391](#) Range constructor for `std::string_view`
- [P1394](#) Range constructor for `std::span`
- [P1664](#) Reconstructible Ranges